

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-a27195cbe412eb8d3.jsonl`
- SHA-256: `0382bdc8260648d7f26295b991a6cf459234d27979c219a13fa0ae964abe0eb`
- Source modified: `2026-06-12T10:43:22+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:37:14.895Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

LENS: correctness + edge cases. Hunt for REAL bugs only (not style):

- ffmpeg/ffprobe flag semantics: does "-fps_mode passthrough" + mp4 mux truly preserve frame count/timestamps? does "-map 0:a?" + "-c:a aac" behave when there is no audio?
- scale=-2:min(H\,ih) escaping inside a list-arg (no shell) on Windows? odd source heights?
- parity logic: nb_read_packets as frame count (B-frames? attached pics? cover art streams selected by v:0?), the duration tolerance math (avg_fps None path), float fps comparison.
- error paths: failed encode leaving a partial file (is it deleted? -y semantics), probe failures mid-flow, manifest written before/after which steps, --force overwrite semantics, exit codes.
- the VFR gate: is_vfr logic vs real camera files (e.g. r=120000/1001 avg=60000/1001), can a VFR source pass parity yet still mislead frame/fps?
- tests: do any tests assert the WRONG behavior or mock away the very thing they claim to test?

2. user (2026-06-12T10:37:20.707Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)

a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)

docs/INGESTION_PIPELINE.md		29	++++-
src/cli/curate.py		147	+++++
src/core/proxy.py		242	+++++
tests/test_proxy.py		309	+++++

4 files changed, 724 insertions(+), 3 deletions(-)

3. user (2026-06-12T10:37:21.687Z)

4. user (2026-06-12T10:37:25.531Z)

```

1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the  $\pm 0.3$  s boundary bar.
7      The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8      downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9      so frame N on the proxy IS frame N on the original and returned labels apply to
10     the original unchanged.
11
12     The ADR-002 proxy contract enforced here:
13     - video timestamps pass through untouched (`-fps_mode passthrough`, never
14       `-r`): same frame count, same fps, no trim, no leading padding;
15     - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16       failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17       golden labels;
18     - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19       survives the handoff (the indexer keys videos by file MD5; downstream
20       processing must keep running on the ORIGINAL file).
21     """
22     import csv
23     import hashlib
24     import os
25     import shutil
26     import subprocess
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional
29
30
31     class ProxyError(RuntimeError):
32         """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35     #: Columns of the provenance manifest (one row appended per generated proxy).
36     MANIFEST_FIELDS = [
37         "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38         "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39         "encoder_settings", "created_at",
40     ]
41
42     # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43     # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44     # back-stepping; CRF 19 keeps the ball/shuttle legible.
45     DEFAULT_HEIGHT = 1080
46     DEFAULT_CRF = 19
47     DEFAULT_GOP = 15
48     DEFAULT_PRESET = "medium"
49
50
51     @dataclass
52     class ProbeInfo:
53         """The frame?time identity of one video stream, as ffprobe reports it."""
54         width: Optional[int] = None
55         height: Optional[int] = None
56         avg_fps: Optional[float] = None      # avg_frame_rate (frames / duration)
57         r_fps: Optional[float] = None        # r_frame_rate (the container's base rate)
58         nb_frames: Optional[int] = None      # demuxed packet count == frame count
59         duration: Optional[float] = None     # video stream duration, else container
60
61         @property

```

```

62     def resolution(self) -> str:
63         if self.width and self.height:
64             return f"{self.width}x{self.height}"
65         return "unknown"
66
67
68     def parse_rate(value: Optional[str]) -> Optional[float]:
69         """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
70         if not value:
71             return None
72         try:
73             if "/" in value:
74                 num, den = value.split("/", 1)
75                 num_f, den_f = float(num), float(den)
76                 if den_f == 0:
77                     return None
78                 return num_f / den_f
79             return float(value)
80         except ValueError:
81             return None
82
83
84     def probe_video(path: str) -> ProbeInfo:
85         """Probe the frame?time identity of `path` via ffprobe (loud on failure).
86
87         Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88         yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89         probe, a failure here RAISES: parity verification is the whole point of the
90         proxy flow, so a missing ffprobe must stop it, not degrade it.
91         """
92         if not shutil.which("ffprobe"):
93             raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
94         if not os.path.exists(path):
95             raise ProxyError(f"video not found: {path}")
96         import json as _json
97         try:
98             out = subprocess.run(
99                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
100                  "-count_packets",
101                  "-show_entries",
102                  "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103                  ":format=duration",
104                  "-of", "json", path],
105                 stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106             ).stdout
107             doc = _json.loads(out)
108         except (subprocess.SubprocessError, OSError, ValueError) as e:
109             raise ProxyError(f"ffprobe failed on {path}: {e}")
110
111         streams = doc.get("streams") or []
112         if not streams:
113             raise ProxyError(f"no video stream found in {path}")
114         s = streams[0]
115
116         def _f(value) -> Optional[float]:
117             try:
118                 return float(value)
119             except (TypeError, ValueError):
120                 return None
121
122         def _i(value) -> Optional[int]:
123             try:
124                 return int(value)
125             except (TypeError, ValueError):
126                 return None

```

```

127
128 # Prefer the video stream's own duration; fall back to the container's
129 # (audio encoder priming can stretch the container a few hundredths).
130 duration = _f(s.get("duration"))
131 if duration is None:
132     duration = _f((doc.get("format") or {}).get("duration"))
133
134 return ProbeInfo(
135     width=_i(s.get("width")),
136     height=_i(s.get("height")),
137     avg_fps=parse_rate(s.get("avg_frame_rate")),
138     r_fps=parse_rate(s.get("r_frame_rate")),
139     nb_frames=_i(s.get("nb_read_packets")),
140     duration=duration,
141 )
142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                        crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                        preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191

```

```

192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
194     """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []
202
203     if src.nb_frames is None or proxy.nb_frames is None:
204         errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
205     elif src.nb_frames != proxy.nb_frames:
206         errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208     if src.avg_fps is None or proxy.avg_fps is None:
209         errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210     elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211         errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213     if src.duration is None or proxy.duration is None:
214         errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215     else:
216         tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217         if abs(proxy.duration - src.duration) > tol:
218             errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                           f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221     return errors
222
223
224 def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225     """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226     digest = hashlib.md5()
227     with open(path, "rb") as f:
228         for chunk in iter(lambda: f.read(chunk_size), b""):
229             digest.update(chunk)
230     return digest.hexdigest()
231
232
233 def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234     """Append one provenance row, writing the header only on first creation."""
235     parent = os.path.dirname(os.path.abspath(manifest_path))
236     os.makedirs(parent, exist_ok=True)
237     is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238     with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239         writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240         if is_new:
241             writer.writeheader()
242         writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

5. user (2026-06-12T10:37:31.137Z)

```

diff --git a/src/cli/curate.py b/src/cli/curate.py
index fde4826..a6d76c2 100644
--- a/src/cli/curate.py
+++ b/src/cli/curate.py
@@ -643,6 +643,120 @@ class SportsDataCLI:
     print(f"Skipped (no annotations) : {skipped_empty}")

```

```

print(f"Manifest      : {manifest_path}\n")

+ def prep_annotation_proxy(self, args):
+     """Re-encode a video into a frame-identical annotation proxy (ADR-002).
+
+     The proxy is what the annotation vendor receives: downscaled and
+     scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
+     but with the source's EXACT frame count / fps / timeline, so frame N
+     on the proxy is frame N on the original and returned labels apply to
+     the original unchanged. Parity is verified after the encode; a proxy
+     that fails verification is deleted, never shared. Provenance (source +
+     proxy MD5s) is appended to the proxy manifest ? downstream processing
+     (import-local, export, the indexer) keeps running on the ORIGINAL file.
+     """
+     from src.core import proxy as proxylib
+
+     src = args.video_path
+     if not os.path.exists(src):
+         print(f"Error: video '{src}' not found.")
+         sys.exit(1)
+
+     stem = os.path.basename(src).split('.')[0]
+     # Mirror import-local's video_id derivation so the manifest row and a
+     # later DB registration of the same source agree on the key.
+     video_id = args.video_id or stem.lower().replace(" ", "_")
+     out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
+     if os.path.abspath(out) == os.path.abspath(src):
+         print("Error: --out must differ from --video-path.")
+         sys.exit(1)
+     if os.path.exists(out) and not args.force:
+         print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
+         sys.exit(1)
+
+     try:
+         src_info = proxylib.probe_video(src)
+     except proxylib.ProxyError as e:
+         print(f"Error: {e}")
+         sys.exit(1)
+     if src_info.avg_fps is None:
+         print(f"Error: could not determine fps of '{src}' ? the frame<->time "
+               f"contract needs it.")
+         sys.exit(1)
+     if proxylib.is_vfr(src_info) and not args.allow_vfr:
+         print(f"Error: '{src}' looks variable-frame-rate "
+               f"r={src_info.r_fps} vs avg={src_info.avg_fps}.")
+         print(" The vendor pipeline recomputes times as frame/fps, which is only "
+               "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
+               "proceed anyway (timestamps stay identical to the original, but "
+               "frame/fps seconds are approximate for BOTH files).")
+         sys.exit(1)
+
+     os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
+     cmd = proxylib.build_proxy_command(src, out, height=args.height,
+                                       crf=args.crf, gop=args.gop,
+                                       preset=args.preset)
+     print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
+           f"f{src_info.avg_fps}g preserved)...")
+     try:
+         proxylib.run_ffmpeg(cmd)
+         proxy_info = proxylib.probe_video(out)
+     except proxylib.ProxyError as e:
+         print(f"Error: {e}")
+         sys.exit(1)
+
+     errors = proxylib.verify_parity(src_info, proxy_info)

```

```

+         if errors:
+             print("Error: proxy FAILED frame<->time parity verification:")
+             for err in errors:
+                 print(f" - {err}")
+             try:
+                 os.remove(out)
+                 print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
+                       f"poisons golden labels).")
+             except OSError:
+                 print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
+                       f"share it with the vendor.")
+             sys.exit(1)
+
+         print("Hashing source + proxy (provenance)...")
+         source_md5 = proxylib.md5_file(src)
+         proxy_md5 = proxylib.md5_file(out)
+
+         from datetime import datetime, timezone
+         proxylib.append_manifest_row(args.manifest, {
+             "video_id": video_id,
+             "source_path": os.path.abspath(src),
+             "source_md5": source_md5,
+             "proxy_path": os.path.abspath(out),
+             "proxy_md5": proxy_md5,
+             "fps": f"{src_info.avg_fps:g}",
+             "frame_count": src_info.nb_frames,
+             "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None else "",
+             "source_resolution": src_info.resolution,
+             "proxy_resolution": proxy_info.resolution,
+             "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
+                                f"preset={args.preset} height<={args.height} "
+                                f"fps_mode=passthrough audio=aac160k"),
+             "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
+         })
+
+         src_mb = os.path.getsize(src) / (1024 * 1024)
+         out_mb = os.path.getsize(out) / (1024 * 1024)
+         print("\n=== Annotation proxy ready ===")
+         print(f" Video ID      : {video_id}")
+         print(f" Source       : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5 {source_md5})")
+         print(f" Proxy        : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5 {proxy_md5})")
+         print(f" Parity       : VERIFIED ? {src_info.nb_frames} frames @ {src_info.avg_fps:g} "
+               f"fps on both")
+         print(f" Manifest     : {args.manifest}")
+         print("\nNext:")
+         print(" 1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
+               "box of every rally on the exact contact/dead frame).")
+         print(" 2. On delivery, convert-cvat may use --video-path with either file "
+               "(identical fps/frame count by construction).")
+         print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
+               "indexer keys videos by its MD5.\n")
+
+     def convert_cvat(self, args):
+         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
+
+@@ -1017,6 +1131,37 @@ def main():
+     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0, metavar="SECONDS",
+                               help="Base backoff between retries; grows linearly per attempt "
+                               "(default: 20)")
+
+     # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
+     prep_parser = subparsers.add_parser(
+         "prep-annotation-proxy",

```

```

+         help="Re-encode a video into a light, frame-identical annotation proxy for "
+         "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "
+         "count/fps/timeline, parity-verified, provenance recorded")
+     prep_parser.add_argument("--video-path", required=True,
+                             help="The ORIGINAL source video (stays the system-of-record
file)")
+     prep_parser.add_argument("--out",
+                             help="Proxy output path (default:
data/videos/proxies/<stem>_proxy.mp4)")
+     prep_parser.add_argument("--video-id",
+                             help="Manifest key (default: derived from the filename, like
import-local)")
+     prep_parser.add_argument("--height", type=int, default=1080,
+                             help="Max proxy height; never upscales (default: 1080 ? "
+                             "720 risks losing the table-tennis ball)")
+     prep_parser.add_argument("--crf", type=int, default=19,
+                             help="x264 quality, lower=better (default: 19)")
+     prep_parser.add_argument("--gop", type=int, default=15,
+                             help="Max keyframe interval in frames; small = snappy "
+                             "frame-stepping (default: 15)")
+     prep_parser.add_argument("--preset", default="medium",
+                             help="x264 speed/size preset (default: medium)")
+     prep_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
+                             help="Provenance CSV to append to (default:
data/annotation_proxy_manifest.csv; "
+                             "commit it ? it is the proxy<->original mapping of record)")
+     prep_parser.add_argument("--allow-vfr", action="store_true",
+                             help="Proceed on a variable-frame-rate source (frame/fps "
+                             "seconds are approximate; default: refuse)")
+     prep_parser.add_argument("--force", action="store_true",
+                             help="Overwrite an existing proxy file")
+
+     # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
+     cvat_parser = subparsers.add_parser(
+         "convert-cvat",
@@ -1077,6 +1222,8 @@ def main():
+     cli.curate()
+     elif args.command == "import-local":
+         cli.import_local(args)
+     elif args.command == "prep-annotation-proxy":
+         cli.prep_annotation_proxy(args)
+     elif args.command == "convert-cvat":
+         cli.convert_cvat(args)
+     elif args.command == "validate-delivery":

```

6. user (2026-06-12T10:37:34.990Z)

```

1     """Tests for the ADR-002 annotation-proxy flow (src/core/proxy.py + the
2     `prep-annotation-proxy` CLI command).
3
4     Everything runs offline: ffmpeg/ffprobe are mocked; only the pure logic
5     (command construction, parity verification, hashing, manifest append) and the
6     CLI orchestration (refusals, deletion-on-failure, provenance row) are exercised.
7     """
8     import csv
9     import os
10    from types import SimpleNamespace
11    from unittest import mock
12
13    import pytest
14    import yaml
15
16    from src.cli.curate import SportsDataCLI
17    from src.core.proxy import (

```



```

18     MANIFEST_FIELDS, ProbeInfo, ProxyError, append_manifest_row,
19     build_proxy_command, is_vfr, md5_file, parse_rate, probe_video,
20     verify_parity,
21 )
22
23
24 # ----- #
25 # Pure logic #
26 # ----- #
27
28 def test_build_command_preserves_the_timeline():
29     cmd = build_proxy_command("in.mp4", "out.mp4")
30     # The ADR-002 invariants: no fps change, no trim, timestamps passed through.
31     assert "-r" not in cmd
32     assert "-ss" not in cmd and "-t" not in cmd and "-to" not in cmd
33     assert cmd[cmd.index("-fps_mode") + 1] == "passthrough"
34     # Scrub-friendly + size knobs at their ratified defaults.
35     assert cmd[cmd.index("-g") + 1] == "15"
36     assert cmd[cmd.index("-crf") + 1] == "19"
37     assert cmd[cmd.index("-vf") + 1] == "scale=-2:min(1080\\,ih)"
38     assert cmd[cmd.index("-movflags") + 1] == "+faststart"
39     # Audio kept (contact sound is the boundary cue), input before output.
40     assert "0:a?" in cmd
41     assert cmd.index("in.mp4") < cmd.index("out.mp4") == len(cmd) - 1
42
43
44 def test_build_command_custom_knobs():
45     cmd = build_proxy_command("a.mp4", "b.mp4", height=720, crf=22, gop=30,
46     preset="fast")
47     assert cmd[cmd.index("-vf") + 1] == "scale=-2:min(720\\,ih)"
48     assert cmd[cmd.index("-crf") + 1] == "22"
49     assert cmd[cmd.index("-g") + 1] == "30"
50     assert cmd[cmd.index("-preset") + 1] == "fast"
51
52 def test_parse_rate():
53     assert parse_rate("30/1") == 30.0
54     assert abs(parse_rate("60000/1001") - 59.94006) < 1e-4
55     assert parse_rate("25") == 25.0
56     assert parse_rate("0/0") is None
57     assert parse_rate("") is None
58     assert parse_rate(None) is None
59     assert parse_rate("garbage") is None
60
61
62 def test_is_vfr():
63     cfr = ProbeInfo(avg_fps=59.94006, r_fps=59.94006)
64     assert not is_vfr(cfr)
65     vfr = ProbeInfo(avg_fps=59.2, r_fps=60.0)
66     assert is_vfr(vfr)
67     # Unknown rates are treated as VFR (conservative: force the explicit flag).
68     assert is_vfr(ProbeInfo(avg_fps=None, r_fps=30.0))
69     assert is_vfr(ProbeInfo(avg_fps=30.0, r_fps=None))
70
71
72 def _info(frames=5737, fps=59.94006, dur=95.72, w=3840, h=2160):
73     return ProbeInfo(width=w, height=h, avg_fps=fps, r_fps=fps,
74     nb_frames=frames, duration=dur)
75
76
77 def test_verify_parity_ok_and_tolerates_audio_priming():
78     src = _info()
79     # Same frames/fps; container a hair longer (AAC priming) ? still parity.
80     proxy = _info(dur=95.76, w=1920, h=1080)
81     assert verify_parity(src, proxy) == []

```

```

82
83
84 def test_verify_parity_catches_frame_count_change():
85     errors = verify_parity(_info(frames=5737), _info(frames=5730, w=1920, h=1080))
86     assert any("frame count changed" in e for e in errors)
87
88
89 def test_verify_parity_catches_fps_change():
90     errors = verify_parity(_info(fps=59.94006), _info(fps=29.97003, w=1920, h=1080))
91     assert any("fps changed" in e for e in errors)
92
93
94 def test_verify_parity_catches_trim():
95     errors = verify_parity(_info(dur=95.72), _info(dur=94.20, w=1920, h=1080))
96     assert any("duration changed" in e for e in errors)
97
98
99 def test_verify_parity_loud_on_unknown_identity():
100     errors = verify_parity(ProbeInfo(), ProbeInfo())
101     assert len(errors) == 3 # frames, fps, duration all unprovable
102
103
104 def test_md5_file(tmp_path):
105     p = tmp_path / "f.bin"
106     p.write_bytes(b"hello world")
107     assert md5_file(str(p)) == "5eb63bbbe01eed093cb22bb8f5acdc3"
108
109
110 def test_append_manifest_header_written_once(tmp_path):
111     manifest = tmp_path / "sub" / "proxies.csv"
112     append_manifest_row(str(manifest), {"video_id": "v1", "source_md5": "a"})
113     append_manifest_row(str(manifest), {"video_id": "v2", "source_md5": "b"})
114     with open(manifest, newline="", encoding="utf-8") as f:
115         rows = list(csv.reader(f))
116         assert rows[0] == MANIFEST_FIELDS
117         assert len(rows) == 3
118         assert rows[1][0] == "v1" and rows[2][0] == "v2"
119
120
121 # ----- #
122 # probe_video (ffprobe mocked) #
123 # ----- #
124
125 _FFPROBE_JSON = """
126 {
127     "streams": [{
128         "width": 3840, "height": 2160,
129         "r_frame_rate": "60000/1001", "avg_frame_rate": "60000/1001",
130         "duration": "95.720000", "nb_read_packets": "5737"
131     }],
132     "format": {"duration": "95.768000"}
133 }
134 """
135
136
137 def test_probe_video_parses_ffprobe_json(tmp_path):
138     p = tmp_path / "v.mp4"
139     p.write_bytes(b"x")
140     with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"), \
141         mock.patch("src.core.proxy.subprocess.run",
142             return_value=SimpleNamespace(stdout=_FFPROBE_JSON)):
143         info = probe_video(str(p))
144         assert (info.width, info.height) == (3840, 2160)
145         assert info.nb_frames == 5737
146         assert abs(info.avg_fps - 59.94006) < 1e-4

```

```

147         assert info.duration == pytest.approx(95.72) # stream duration preferred
148         assert info.resolution == "3840x2160"
149
150
151     def test_probe_video_requires_ffprobe(tmp_path):
152         p = tmp_path / "v.mp4"
153         p.write_bytes(b"x")
154         with mock.patch("src.core.proxy.shutil.which", return_value=None):
155             with pytest.raises(ProxyError, match="ffprobe not found"):
156                 probe_video(str(p))
157
158
159     def test_probe_video_missing_file():
160         with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"):
161             with pytest.raises(ProxyError, match="not found"):
162                 probe_video("nope.mp4")
163
164
165     def test_probe_video_no_video_stream(tmp_path):
166         p = tmp_path / "v.mp4"
167         p.write_bytes(b"x")
168         with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"), \
169             mock.patch("src.core.proxy.subprocess.run",
170                 return_value=SimpleNamespace(stdout='{ "streams": [] }')):
171             with pytest.raises(ProxyError, match="no video stream"):
172                 probe_video(str(p))
173
174
175     # ----- #
176     # CLI: prep-annotation-proxy #
177     # ----- #
178
179     @pytest.fixture
180     def cli(tmp_path):
181         cfg = {
182             "database_path": str(tmp_path / "test.db"),
183             "supported_sports": ["badminton", "pickleball", "tabletennis"],
184             "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "Proprietary"],
185         }
186         cfg_path = tmp_path / "config.yaml"
187         cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
188         return SportsDataCLI(config_path=str(cfg_path))
189
190
191     def _prep_args(tmp_path, **kw):
192         base = dict(
193             video_path=str(tmp_path / "Match One.mp4"),
194             out=str(tmp_path / "proxies" / "match_proxy.mp4"),
195             video_id=None, height=1080, crf=19, gop=15, preset="medium",
196             manifest=str(tmp_path / "proxy_manifest.csv"),
197             allow_vfr=False, force=False,
198         )
199         base.update(kw)
200         return SimpleNamespace(**base)
201
202
203     def _cfr(frames=100, w=3840, h=2160):
204         return _info(frames=frames, dur=100 / 59.94006, w=w, h=h)
205
206
207     def test_prep_proxy_happy_path(cli, tmp_path):
208         args = _prep_args(tmp_path)
209         (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
210
211         def fake_ffmpeg(cmd):

```

```

212         with open(cmd[-1], "wb") as f:
213             f.write(b"proxy-bytes")
214
215     probes = [_cfr(), _cfr(w=1920, h=1080)] # source probe, then proxy probe
216     with mock.patch("src.core.proxy.probe_video", side_effect=probes), \
217         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
218         cli.prep_annotation_proxy(args)
219
220     assert os.path.exists(args.out)
221     with open(args.manifest, newline="", encoding="utf-8") as f:
222         rows = list(csv.DictReader(f))
223     assert len(rows) == 1
224     row = rows[0]
225     # video_id derived like import-local: stem lowercased, spaces -> underscores.
226     assert row["video_id"] == "match_one"
227     assert row["source_md5"] == md5_file(str(tmp_path / "Match One.mp4"))
228     assert row["proxy_md5"] == md5_file(args.out)
229     assert row["source_md5"] != row["proxy_md5"]
230     assert row["fps"].startswith("59.94")
231     assert row["frame_count"] == "100"
232     assert row["source_resolution"] == "3840x2160"
233     assert row["proxy_resolution"] == "1920x1080"
234
235
236 def test_prep_proxy_parity_failure_deletes_proxy(cli, tmp_path):
237     args = _prep_args(tmp_path)
238     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
239
240     def fake_ffmpeg(cmd):
241         with open(cmd[-1], "wb") as f:
242             f.write(b"proxy-bytes")
243
244     # Proxy comes back one frame short -> parity check must fail loudly.
245     probes = [_cfr(frames=100), _cfr(frames=99, w=1920, h=1080)]
246     with mock.patch("src.core.proxy.probe_video", side_effect=probes), \
247         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
248         with pytest.raises(SystemExit) as exc:
249             cli.prep_annotation_proxy(args)
250     assert exc.value.code == 1
251     assert not os.path.exists(args.out), "a failed proxy must never be left on disk"
252     assert not os.path.exists(args.manifest), "no provenance row for a failed proxy"
253
254
255 def test_prep_proxy_refuses_vfr_without_flag(cli, tmp_path):
256     args = _prep_args(tmp_path)
257     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
258     vfr_info = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
259                          nb_frames=100, duration=1.69)
260     with mock.patch("src.core.proxy.probe_video", return_value=vfr_info):
261         with pytest.raises(SystemExit) as exc:
262             cli.prep_annotation_proxy(args)
263     assert exc.value.code == 1
264     assert not os.path.exists(args.out)
265
266
267 def test_prep_proxy_allow_vfr_proceeds(cli, tmp_path):
268     args = _prep_args(tmp_path, allow_vfr=True)
269     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
270     vfr = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
271                    nb_frames=100, duration=100 / 59.2)
272     vfr_proxy = ProbeInfo(width=1920, height=1080, avg_fps=59.2, r_fps=60.0,
273                          nb_frames=100, duration=100 / 59.2)
274
275     def fake_ffmpeg(cmd):
276         with open(cmd[-1], "wb") as f:

```

```

277         f.write(b"proxy-bytes")
278
279     with mock.patch("src.core.proxy.probe_video", side_effect=[vfr, vfr_proxy]), \
280         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
281         cli.prep_annotation_proxy(args)
282     assert os.path.exists(args.out)
283
284
285     def test_prep_proxy_refuses_existing_output(cli, tmp_path):
286         args = _prep_args(tmp_path)
287         (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
288         os.makedirs(os.path.dirname(args.out), exist_ok=True)
289         with open(args.out, "wb") as f:
290             f.write(b"existing")
291         with pytest.raises(SystemExit) as exc:
292             cli.prep_annotation_proxy(args)
293         assert exc.value.code == 1
294
295
296     def test_prep_proxy_missing_source(cli, tmp_path):
297         args = _prep_args(tmp_path, video_path=str(tmp_path / "nope.mp4"))
298         with pytest.raises(SystemExit) as exc:
299             cli.prep_annotation_proxy(args)
300         assert exc.value.code == 1
301
302
303     def test_prep_proxy_out_equals_source_refused(cli, tmp_path):
304         src = tmp_path / "Match One.mp4"
305         src.write_bytes(b"original-bytes")
306         args = _prep_args(tmp_path, out=str(src))
307         with pytest.raises(SystemExit) as exc:
308             cli.prep_annotation_proxy(args)
309         assert exc.value.code == 1
310

```

7. assistant (2026-06-12T10:39:13.469Z)

Now let me check the `import-local` video_id derivation (the CLI comment claims to mirror it) and the CVAT parser's frame/fps recompute.

8. user (2026-06-12T10:39:15.265Z)

```

63:     def _guard_import(self, sport_str, video_id, annotations, force=False):
64:         # annotations (filename-derived video_ids collide easily).
65:         if self.db.get_video(video_id):
66:             n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
67:             print(f"Error: video_id '{video_id}' already has {n} annotated segment(s); "
68:                 f"Warning: --force given ? replacing {n} existing segment(s) for "
69:                 f"video_id '{video_id}'.")
70:             match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
71:             video_id = args.video_id or match_name.lower().replace(" ", "_")
72:             annotations = self._parse_local_annotations(sport_str, video_id,
73:                 annotations_path)
74:             self._guard_import(sport_str, video_id, annotations, force=getattr(args, "force",
75:                 False))
76:             video_id=video_id,
77:             print(f" - Video ID: {video_id}")
78:             src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
79:             "unknown"
80:
81:     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
82:     List[Any]:
83:         ext = os.path.splitext(filepath)[1].lower()
84:         video_id=video_id,
85:         video_id=video_id,
86:         video_id=video_id,
87:

```

```

270:             video_id=video_id,
282:             video_id=video_id,
301:             video_id=video_id,
313:             video_id=video_id,
324:             video_id=video_id,
337:             video_id=video_id,
349:             video_id=video_id,
436:         if args.video_id:
437:             videos = [v for v in videos if v.video_id == args.video_id]
443:         results = [] # (video_id, action, detail)
446:             results.append((video.video_id, "skip", "non-commercial"))
451:             results.append((video.video_id, "skip", "no real source URL"))
454:             dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
460:             results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
463:             print(f"Fetching {video.video_id} ({video.match_name})...")
467:             results.append((video.video_id, "FAIL", fail_reason))
483:             results.append((video.video_id, "fetched", detail))
520:             if video.video_id in seen:
522:                 seen.add(video.video_id)
535:             print(f" {v.video_id:40s} {v.license_type.value} (video) -> annotation_license;
"

551:         Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
555:         indexer keys videos by *file MD5*, not by our `video_id`, so it must run
590:             intervals = self.db.get_segment_intervals(video.video_id, sport)
597:             csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
605:             "video_id": video.video_id,
633:             summary = [[m["sport"], m["video_id"], m["num_segments"],
665:             stem = os.path.basename(src).split('.')[0]
666:             # Mirror import-local's video_id derivation so the manifest row and a
668:             video_id = args.video_id or stem.lower().replace(" ", "_")
669:             out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
728:             "video_id": video_id,
747:             print(f" Video ID      : {video_id}")
791:             src_name = os.path.splitext(os.path.basename(args.input))[0]
828:             vid = args.video_id or "delivery"
903:             video.video_id,
934:             rallies = self.db.get_badminton_rallies(video.video_id)
936:             rallies = self.db.get_pickleball_rallies(video.video_id)
938:             rallies = self.db.get_tabletennis_rallies(video.video_id)
940:             pts = self.db.get_tennis_points(video.video_id)
942:             delivs = self.db.get_cricket_deliveries(video.video_id)
947:             print(f"Video ID      : {video.video_id}")
981:             self.db.update_status(video.video_id, CurationStatus.CURATED)

```

[Showing results with pagination = limit: 60]

9. user (2026-06-12T10:39:16.278Z)

```

7:instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
17:1. **Recompute time from the frame range.** A rally's interval is taken from the
18: first/last *frame* a box for that rally appears on:
19: ``seconds = frame / fps`` at the video's true fps. We **ignore** any
21: samples every ``step`` frames and assumes 30 fps yields systematically wrong
22: times (we have seen ``start_time == frame / (step * 30)``, i.e. off by the
23: sampling step at the wrong fps). The frame index is reliable; the vendor's
32:(``<image><box>...``) or **track/interpolation mode** (``<track><box frame=...>``).
47:from src.core.downloader import probe_resolution_fps
107:     """One rally recovered from the CVAT export, time re-derived from frames."""
109:     start_time: float           # seconds, = frame_min / fps
110:     end_time: float             # seconds, = frame_max / fps (see padding note)
117:     frame_min: int
118:     frame_max: int
141:     fps: float
168:     ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one

```

```

172:     ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
196:         stop = job.findtext("stop_frame")
198:         meta["stop_frame"] = int(stop.strip())
199:         ff = job.findtext("frame_filter") or ""
214:     # Image mode: <image id=frame> ... <box>...</box>
217:         frame = int(img.get("id"))
221:         boxes.append({"frame": frame, "attrs": _attrs(box)})
223:     # Track mode: <track> ... <box frame=N outside=0>...</box>
227:         continue # a track's terminating keyframe carries no real label
229:         frame = int(box.get("frame"))
232:         boxes.append({"frame": frame, "attrs": _attrs(box)})
237: def _infer_step(frames: List[int]) -> int:
238:     """Most common positive gap between consecutive distinct sampled frames."""
239:     uniq = sorted(set(frames))
246: def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
247:     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
248:     if fps is None or fps <= 0:
249:         raise ValueError("fps must be a positive number to convert frames to seconds")
261:     frames = sorted(b["frame"] for b in items)
263:     fmin, fmax = frames[0], frames[-1]
266:     start_time=fmin / fps,
267:     end_time=fmax / fps,
274:     frame_min=fmin,
275:     frame_max=fmax,

```

[Showing results with pagination = limit: 40]

10. user (2026-06-12T10:39:31.137Z)

```

112: def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int], Optional[float]]:
113:     """Return (width, height, fps) of a video file via ffprobe, best-effort.
114:
115:     Returns (None, None, None) if ffprobe is unavailable or the file can't be
116:     parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
117:     """
118:     if not shutil.which("ffprobe") or not os.path.exists(path):
119:         return (None, None, None)
120:     try:
121:         out = subprocess.run(
122:             ["ffprobe", "-v", "error", "-select_streams", "v:0",
123:              "-show_entries", "stream=width,height,avg_frame_rate",
124:              "-of", "default=noprint_wrappers=1:nokey=1", path],
125:             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
126:         ).stdout.split()
127:     except (subprocess.SubprocessError, OSError):
128:         return (None, None, None)
129:
130:     width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
131:     height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
132:     fps = None
133:     if len(out) > 2 and "/" in out[2]:
134:         num, den = out[2].split("/")
135:         try:
136:             den_f = float(den)
137:             fps = round(float(num) / den_f, 3) if den_f else None
138:         except ValueError:
139:             fps = None
140:     return (width, height, fps)
141:
142:
143: @dataclass
144: class DownloadResult:
145:     success: bool
146:     path: Optional[str] = None
147:     size_mb: float = 0.0

```

```
148- width: Optional[int] = None
149- height: Optional[int] = None
150- fps: Optional[float] = None
151- resolution_label: Optional[str] = None
152- error: Optional[str] = None
```

11. user (2026-06-12T10:39:32.205Z)

```
43-from dataclasses import dataclass, field
44-from typing import Any, Dict, List, Optional, Tuple
45-
46-from src.core.parsing import safe_int
47:from src.core.downloader import probe_resolution_fps
48-
49-# The agreed cross-sport ending_reason vocabulary for net-separated sports.
50-# Kept here as a soft check only (a warning, never a hard failure) so this
51-# adapter does not depend on the annotation-spec wording.
52-
53-
54-
55-
56-
57-
58-
59-
60-
61-
62-
63-
64-
65-
66-
67-
68-
69-
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-
141-
142-
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-def convert(xml_path: str,
368-            fps: Optional[float] = None,
369-            video_path: Optional[str] = None,
370-            step: Optional[int] = None,
371-            allowed_reasons: Optional[set] = None) -> ConversionResult:
372-    """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
373-
374-    ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
375-    the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
376-    that silent guess is exactly the bug this adapter exists to undo.
377-
378-    """
379-    meta, boxes = parse_cvat_boxes(xml_path)
380-
381-    if fps is None and video_path:
382-        _, _, probed = probe_resolution_fps(video_path)
383-        fps = probed
384-
385-    if fps is None or fps <= 0:
386-        raise ValueError(
387-            "Could not determine fps. Pass fps=... or a readable video_path so "
388-            "frames can be converted to seconds (seconds = frame / fps).")
389-
390-    if step is None:
391-        step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
```

12. assistant (2026-06-12T10:41:22.181Z)

Let me get exact line numbers in the new `curate.py` for the findings, and verify the encode-failure path.

13. user (2026-06-12T10:41:23.069Z)

```
646 def prep_annotation_proxy(self, args):
647     """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648
649     The proxy is what the annotation vendor receives: downscaled and
650     scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651     but with the source's EXACT frame count / fps / timeline, so frame N
652     on the proxy is frame N on the original and returned labels apply to
653     the original unchanged. Parity is verified after the encode; a proxy
654     that fails verification is deleted, never shared. Provenance (source +
655     proxy MD5s) is appended to the proxy manifest ? downstream processing
656     (import-local, export, the indexer) keeps running on the ORIGINAL file.
657     """
658     from src.core import proxy as proxylib
```



```

659
660 src = args.video_path
661 if not os.path.exists(src):
662     print(f"Error: video '{src}' not found.")
663     sys.exit(1)
664
665 stem = os.path.basename(src).split('.')[0]
666 # Mirror import-local's video_id derivation so the manifest row and a
667 # later DB registration of the same source agree on the key.
668 video_id = args.video_id or stem.lower().replace(" ", "_")
669 out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670 if os.path.abspath(out) == os.path.abspath(src):
671     print("Error: --out must differ from --video-path.")
672     sys.exit(1)
673 if os.path.exists(out) and not args.force:
674     print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675     sys.exit(1)
676
677 try:
678     src_info = proxylib.probe_video(src)
679 except proxylib.ProxyError as e:
680     print(f"Error: {e}")
681     sys.exit(1)
682 if src_info.avg_fps is None:
683     print(f"Error: could not determine fps of '{src}' ? the frame<->time "
684           f"contract needs it.")
685     sys.exit(1)
686 if proxylib.is_vfr(src_info) and not args.allow_vfr:
687     print(f"Error: '{src}' looks variable-frame-rate "
688           f"r={src_info.r_fps} vs avg={src_info.avg_fps}.")
689     print(" The vendor pipeline recomputes times as frame/fps, which is only "
690           "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691           "proceed anyway (timestamps stay identical to the original, but "
692           "frame/fps seconds are approximate for BOTH files).")
693     sys.exit(1)
694
695 os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
696 cmd = proxylib.build_proxy_command(src, out, height=args.height,
697                                    crf=args.crf, gop=args.gop,
698                                    preset=args.preset)
699 print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
700       f"fps {src_info.avg_fps:g} preserved)...")
701 try:
702     proxylib.run_ffmpeg(cmd)
703     proxy_info = proxylib.probe_video(out)
704 except proxylib.ProxyError as e:
705     print(f"Error: {e}")
706     sys.exit(1)
707
708 errors = proxylib.verify_parity(src_info, proxy_info)
709 if errors:
710     print("Error: proxy FAILED frame<->time parity verification:")
711     for err in errors:
712         print(f" - {err}")
713     try:
714         os.remove(out)
715         print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
716               f"poisons golden labels).")
717     except OSError:
718         print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
719               f"share it with the vendor.")
720     sys.exit(1)
721
722 print("Hashing source + proxy (provenance)...")
723 source_md5 = proxylib.md5_file(src)

```

```

724         proxy_md5 = proxylib.md5_file(out)
725
726     from datetime import datetime, timezone
727     proxylib.append_manifest_row(args.manifest, {
728         "video_id": video_id,
729         "source_path": os.path.abspath(src),
730         "source_md5": source_md5,
731         "proxy_path": os.path.abspath(out),
732         "proxy_md5": proxy_md5,
733         "fps": f"{src_info.avg_fps:g}",
734         "frame_count": src_info.nb_frames,
735         "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
736     else "",
737         "source_resolution": src_info.resolution,
738         "proxy_resolution": proxy_info.resolution,
739         "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
740                             f"preset={args.preset} height<={args.height} "
741                             f"fps_mode=passthrough audio=aac160k"),
742         "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
743     })
744
745     src_mb = os.path.getsize(src) / (1024 * 1024)
746     out_mb = os.path.getsize(out) / (1024 * 1024)
747     print("\n=== Annotation proxy ready ===")
748     print(f"  Video ID      : {video_id}")
749     print(f"  Source          : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5 {source_md5})")
750     print(f"  Proxy           : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5 {proxy_md5})")
751     print(f"  Parity          : VERIFIED ? {src_info.nb_frames} frames @ {src_info.avg_fps:g} fps on both")
752     print(f"  Manifest        : {args.manifest}")
753     print("\nNext:")
754     print("  1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
755         "box of every rally on the exact contact/dead frame).")
756     print("  2. On delivery, convert-cvat may use --video-path with either file "
757         "(identical fps/frame count by construction).")
758     print("  3. import-local / export keep pointing at the ORIGINAL file ? the "
759         "indexer keys videos by its MD5.\n")
760
761     def convert_cvat(self, args):
762         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
763
764         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
765         attributes) on the ingestion side: each rally's time is recomputed from
766         its frame range at the video's true fps, and content defects (overlaps,

```

14. user (2026-06-12T10:43:15.222Z)

Structured output provided successfully

15. assistant (2026-06-12T10:43:22.677Z)

Review complete. Seven findings returned via structured output: one high (encode-failure paths leave a partial proxy on disk, contradicting the module's own "deleted, never shared" invariant ? and untestable as written since no cleanup exists), two medium (case-insensitive same-file guard on Windows that can destroy the source with `--force`; container-vs-stream duration asymmetry that spuriously deletes good proxies from MKV/TS sources), and four low (ZeroDivisionError on `avg\_fps=0` with `--allow-vfr`, `is\_vfr` false-positives on $r=2\times\text{avg}$ CFR camera files, odd-height libx264 failures, and `v:0` selectors matching cover-art streams). No tests assert wrong behavior; the filter escaping, `map 0:a?`, and the convert-cvat "either file" fps claim all checked out.